

Sharif University of Technology
Department of Mechanical Engineering

Trajectory Planning for the Motion of a Spherical (RRP) Robot

Robotics Course – Homework 5

Haniyeh Zahra Budaqi – 99105956
Instructor: Dr. Behzadipour

Term 1, 1402 (Fall 2023)

Contents

1	Problem Statements	2
1.1	Question 1	2
1.2	Question 2	2
2	Answers to the Questions	3
2.1	Question 1	3
2.2	Question 2	9

1 Problem Statements

The RRP robot model in this problem is completely analogous to the robot model used in the Homework 3 problem. Therefore, the models and code from that homework will be needed here as well.

1.1 Question 1

In the first part of the problem, we are given two points, and it is required that – given these points (which are the start and end coordinates of the end-effector’s motion) – a polynomial trajectory be designed for the joints. The points to keep in mind are continuity of velocity and acceleration and smoothness of the robot’s motion. It is then required that the designed trajectory be applied to the robot in Simulink, and that the joint motion curves and the end-effector coordinates be plotted versus time.

The start and end points of the end-effector are as follows.

$X_i (mm)$	$Y_i (mm)$	$Z_i (mm)$	$X_f (mm)$	$Y_f (mm)$	$Z_f (mm)$
284.829	-164.446	190.293	91.203	340.373	605.682

1.2 Question 2

In this part, it is assumed that the end-effector moves along a straight line, and it is required that the joint trajectories be designed based on a trapezoidal velocity profile for the speed of the end-effector. The designed trajectories must be applied to the model with the help of the Signal Builder block, and the joint motion curves and the end-effector coordinates must be plotted versus time.

$$(L_1 = 31 \text{ cm} , L_2 = 30 \text{ cm})$$

$$(L_1 = 31 \text{ cm} , L_2 = 30 \text{ cm})$$

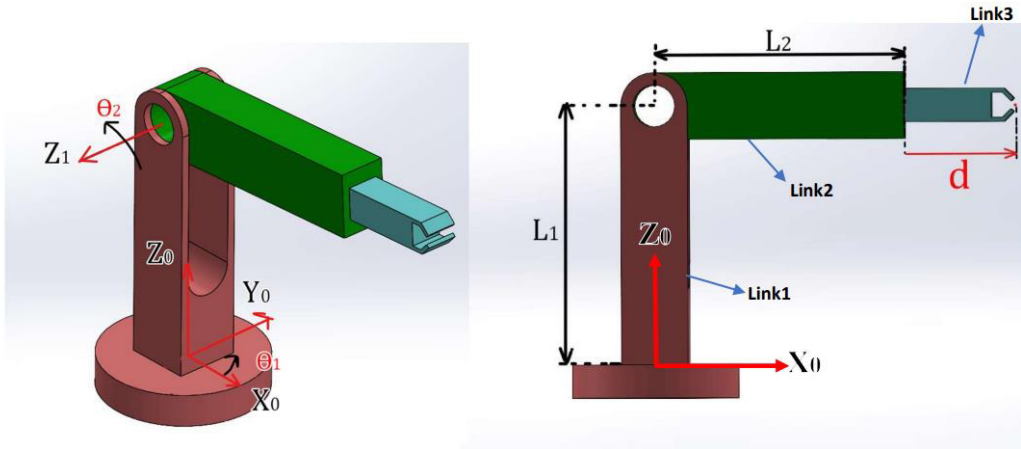


Figure 1: RRP robot model (SolidWorks assembly) – isometric view showing the joint axes Z_0 , Z_1 and the joint variables θ_1 , θ_2 , d ; side view showing Link 1, Link 2, Link 3 and the link lengths L_1 , L_2 .

2 Answers to the Questions

2.1 Question 1

First, we fix the issues from Homework 3. We apply the zero coordinate frame to Link 1 as shown in the figure below:

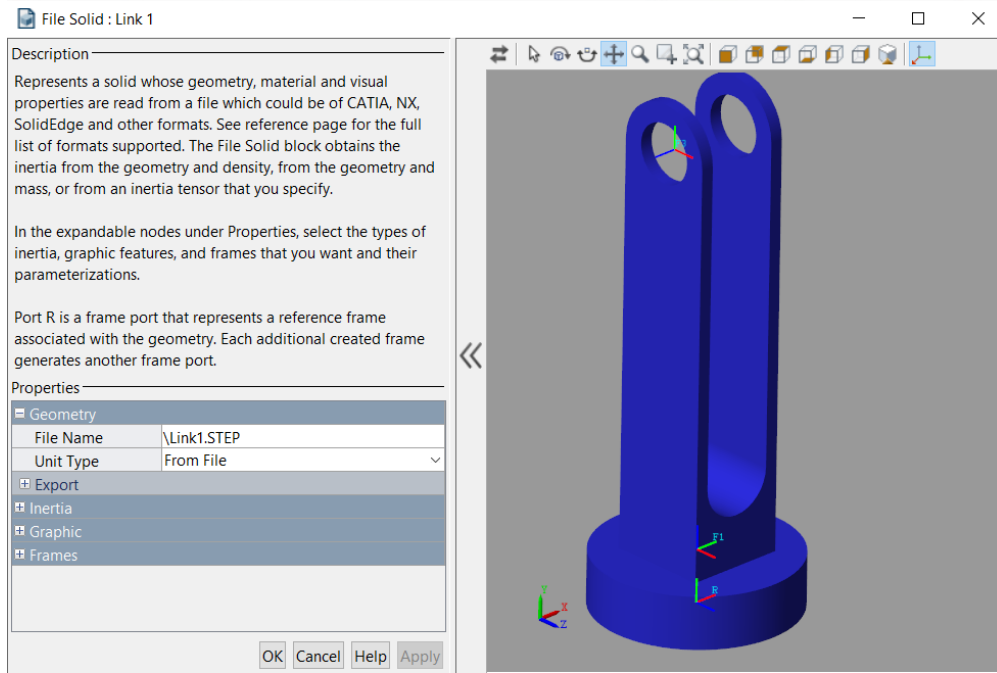


Figure 2: The Simscape “File Solid” block for Link 1, showing the STEP geometry imported from SolidWorks (*Link1.STEP*) with the reference frame (port R) placed at the base of the link.

Then, we correct the code of the *InvKin* function. Given the final end-effector coordinates in H_3^0 as described below, we have:

$$\overrightarrow{O_0O_3} = \begin{bmatrix} \cos \theta_2 \cos \theta_1 * (d + 300) \\ \cos \theta_2 \sin \theta_1 * (d + 300) \\ \sin \theta_2 * (d + 300) + 310 \end{bmatrix}$$

$$\theta_1 = \text{atan} \left(\frac{y}{x} \right)$$

$$d = \sqrt{x^2 + y^2 + (z - 310)^2} - 300$$

$$\theta_2 = \text{acos} \left(\frac{x}{\cos \theta_1 * (d + 300)} \right)$$

Therefore, the *InvKin* function (used to obtain two sets of solutions) becomes as follows:

```
function [theta1_1,theta1_2,theta2_1, theta2_2 ,d1, d2] = InvKin(x,y,z)

theta1_1 = (180/pi)*atan(y/x);
theta1_2 = 180+theta1_1;
```

```

c = sqrt(x^2+y^2+(z-310)^2); %c = d + 300

d1 = c - 300;
d2 = -c - 300;

b = x / (cosd(theta1_1)*(d1+300)); %b = cos theta2

theta2_1 = (180/pi)*acos(b);
theta2_2 = -theta2_1;

end

```

Now, using this function, we obtain the joint variables at the start and end coordinates of the end-effector:

$$X_i = [284.829; -164.446; 190.293]$$

$$X_f = [90.203; 340.373; 605.682]$$

Mode	1	2
$x(mm)$	284.829	90.203
$y(mm)$	-164.446	340.373
$z(mm)$	190.293	605.682
θ_{11}	-30	75
θ_{12}	150	255
θ_{21}	20	40
θ_{22}	-20	-40
d_1	50	160
d_2	-650	-760

We select the solution set $[\theta_{11}, \theta_{21}, d_1]$ from each mode, giving:

Case 1:

$$[\theta_{11}, \theta_{21}, d_1] = [-30, 20, 50]$$

Case 2:

$$[\theta_{11}, \theta_{21}, d_1] = [75, 40, 160]$$

Now, for each joint we design a cubic (multi-term) polynomial motion trajectory:

$$T = 1\text{ s}$$

Joint 1:

$$\theta_1(0) = -30^\circ \quad \dot{\theta}_1(0) = 0$$

$$\theta_1(1) = 75^\circ \quad \dot{\theta}_1(1) = 0$$

$$\theta_1(t) = a_0 + a_1t + a_2t^2 + a_3t^3$$

$$a_0 = \theta_1(0) = -30^\circ$$

$$a_1 = \dot{\theta}_1(0) = 0$$

$$\begin{cases} 75 = -30 + a_2 + a_3 \\ 0 = 2a_2 + 3a_3 \end{cases} \implies a_2 = 315, a_3 = -210$$

$$\theta_1(t) = -30 + 315t^2 - 210t^3$$

By performing the same operations for joints 2 and 3, we also obtain:

$$\theta_2(t) = 20 + 60t^2 - 40t^3$$

$$d(t) = 50 + 330t^2 - 220t^3$$

Using the code below, we generate and store the data for each joint in the workspace:

```
t = 0:0.005:1;
theta1 = (-30 + 315*t.^2 - 210*t.^3)*(pi/180);
theta2 = (20 + 60*t.^2 - 40*t.^3)*(pi/180);
d = (50 + 330*t.^2 - 220*t.^3)/1000;
```

Then, in Simulink, we use the Signal Builder block to construct the signals for each joint:

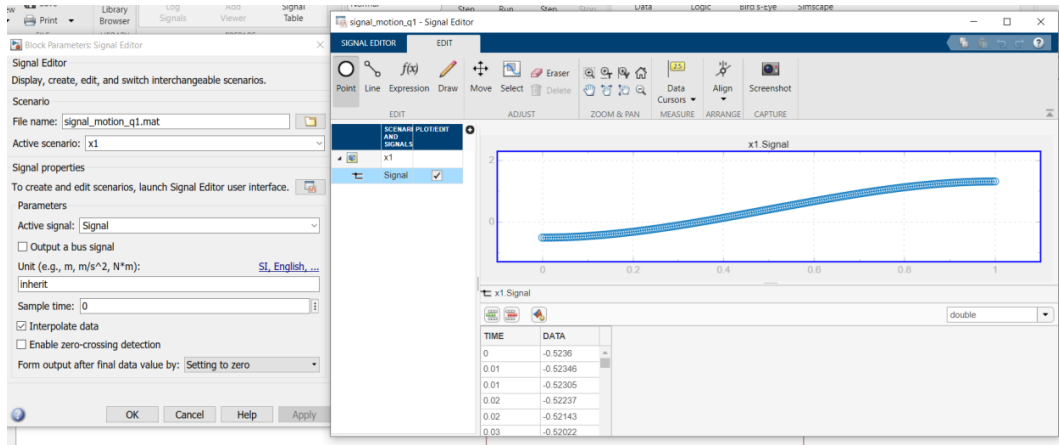


Figure 3: Signal Editor – scenario “x1”, joint 1 (θ_1) trajectory versus time (rad vs. s), rising from -0.524 to about 1.31 rad (-30° to 75°).

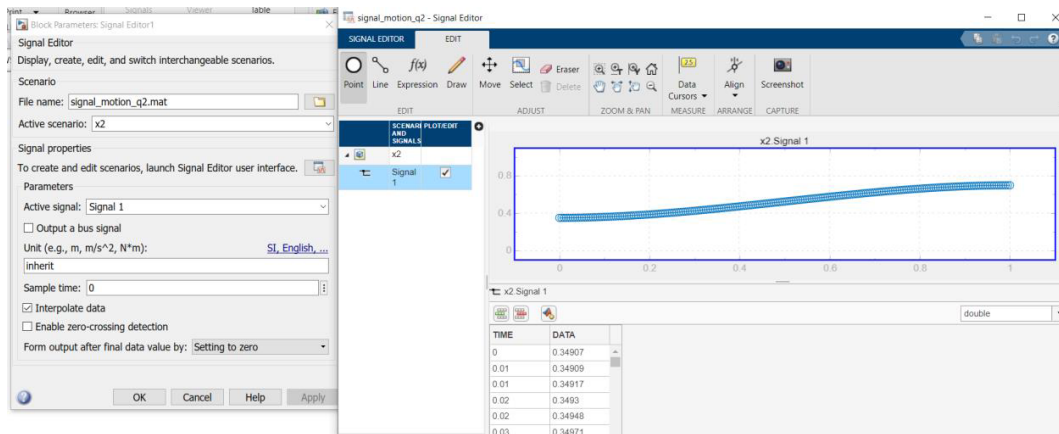


Figure 4: Signal Editor – scenario “x2”, joint 2 (θ_2) trajectory versus time (rad vs. s), rising from 0.349 to about 0.698 rad (20° to 40°).

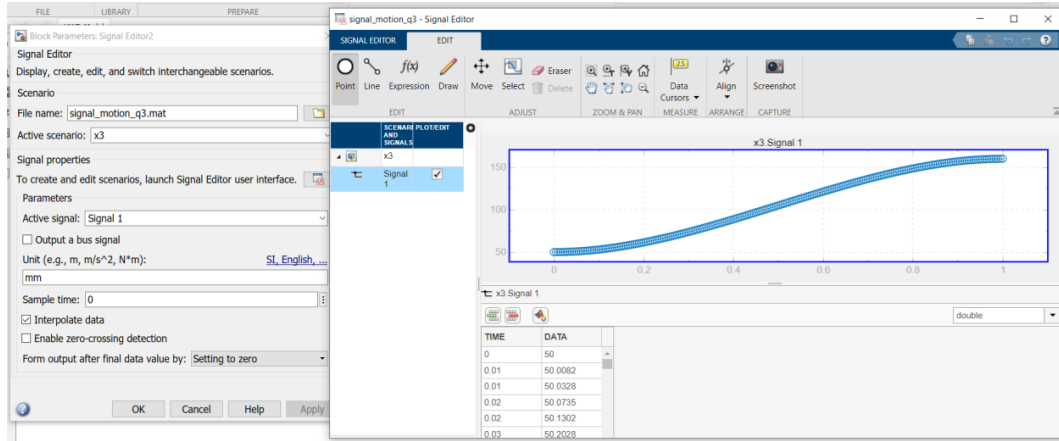


Figure 5: Signal Editor – scenario “x3”, prismatic joint (d) trajectory versus time (mm vs. s), rising from 50 mm to about 160 mm.

Using the following settings in the Simulink-PS Converter, we connect each signal to its corresponding motion input:

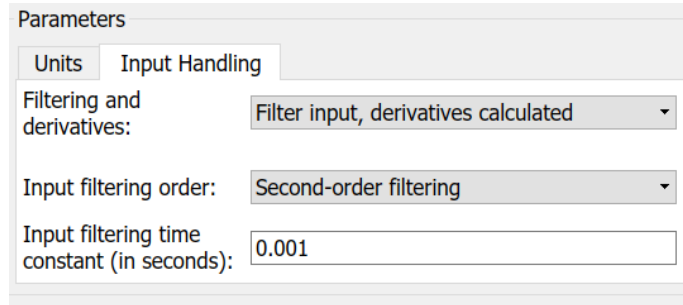


Figure 6: Simulink-PS Converter “Input Handling” settings used for each motion input.

(Filtering and derivatives: “Filter input, derivatives calculated”; Input filtering order: second-order filtering; Input filtering time constant: 0.001 s.)

First, by checking “Position” in the Sensing menu of each joint, we take its position as an output, and then we plot these signals using a Scope block:

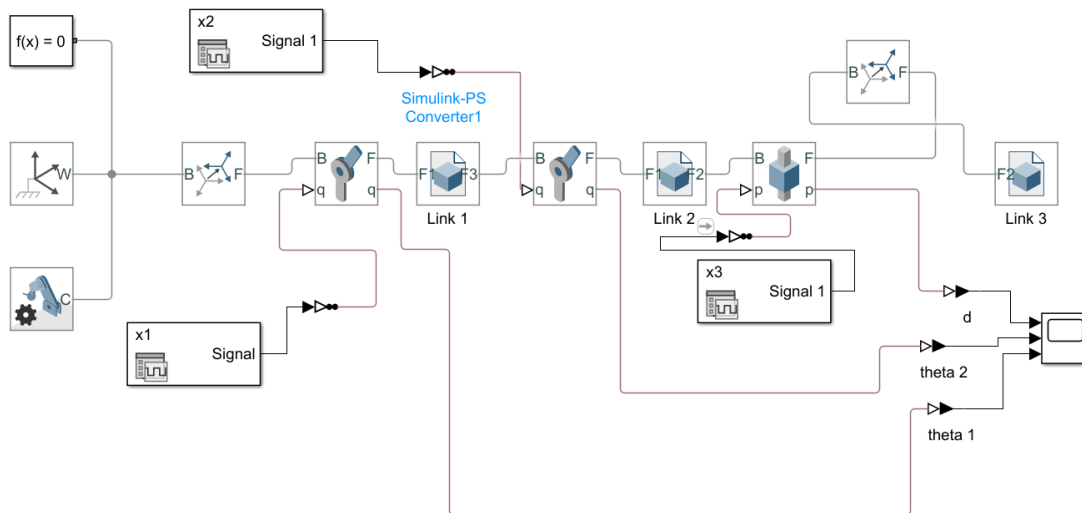


Figure 7: *HW5_Model_1* block diagram – the Signal Editor blocks (x_1 , x_2 , x_3) drive Joint 1, Joint 2 and the prismatic joint (through Simulink-PS Converters) connecting World, Link 1, Link 2 and Link 3; the joint positions (θ_1 , θ_2 , d) are sensed and sent to a Scope.

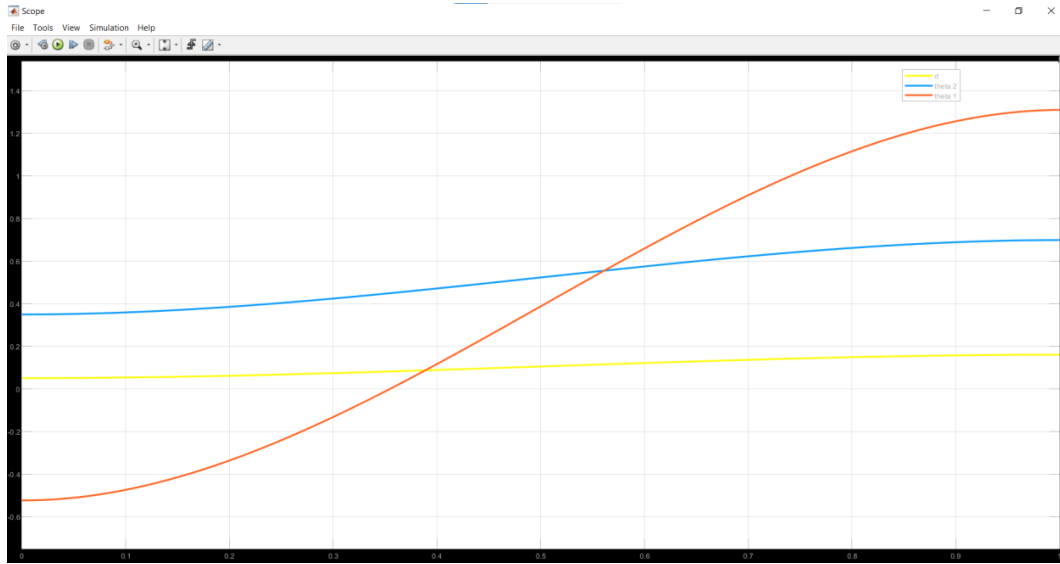


Figure 8: Scope output for Question 1 – joint trajectories θ_1 (orange), θ_2 (blue) and d (yellow) versus time (s), reproducing the polynomial trajectories designed above.

In order to plot the end-effector coordinates as well, we also use a Transform Sensor block and route its X , Y , Z outputs to a separate Scope:

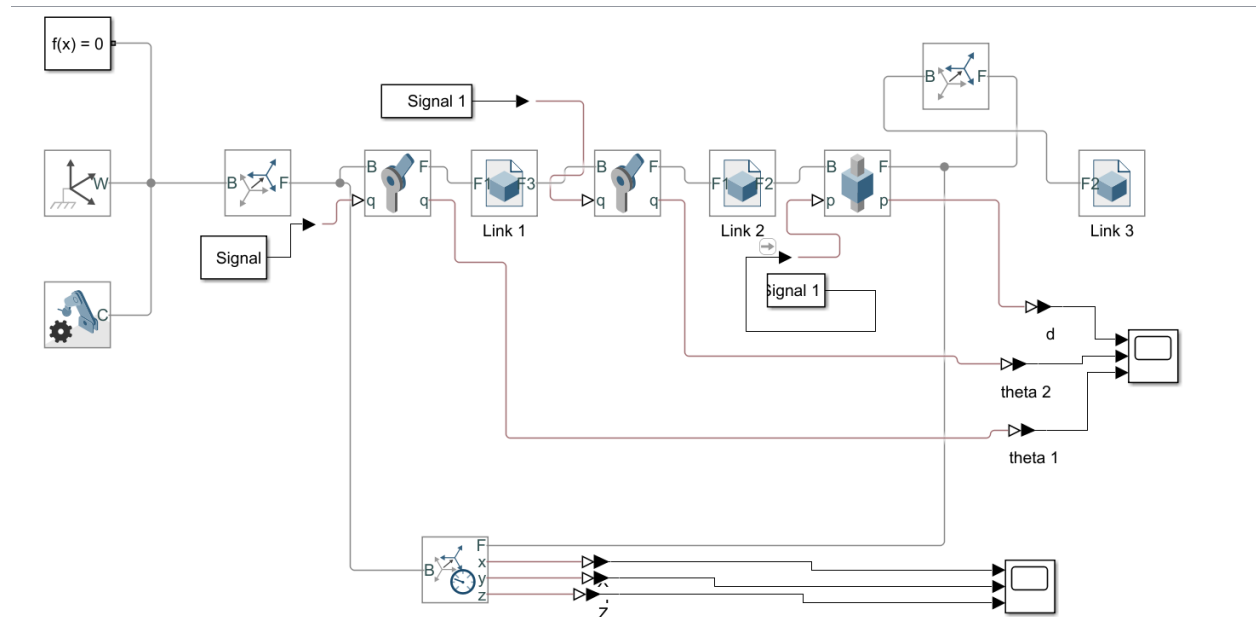


Figure 9: *HW5_Model_1* block diagram extended with a Transform Sensor block, whose F , x , y , z outputs are sent to a second Scope to record the end-effector position.

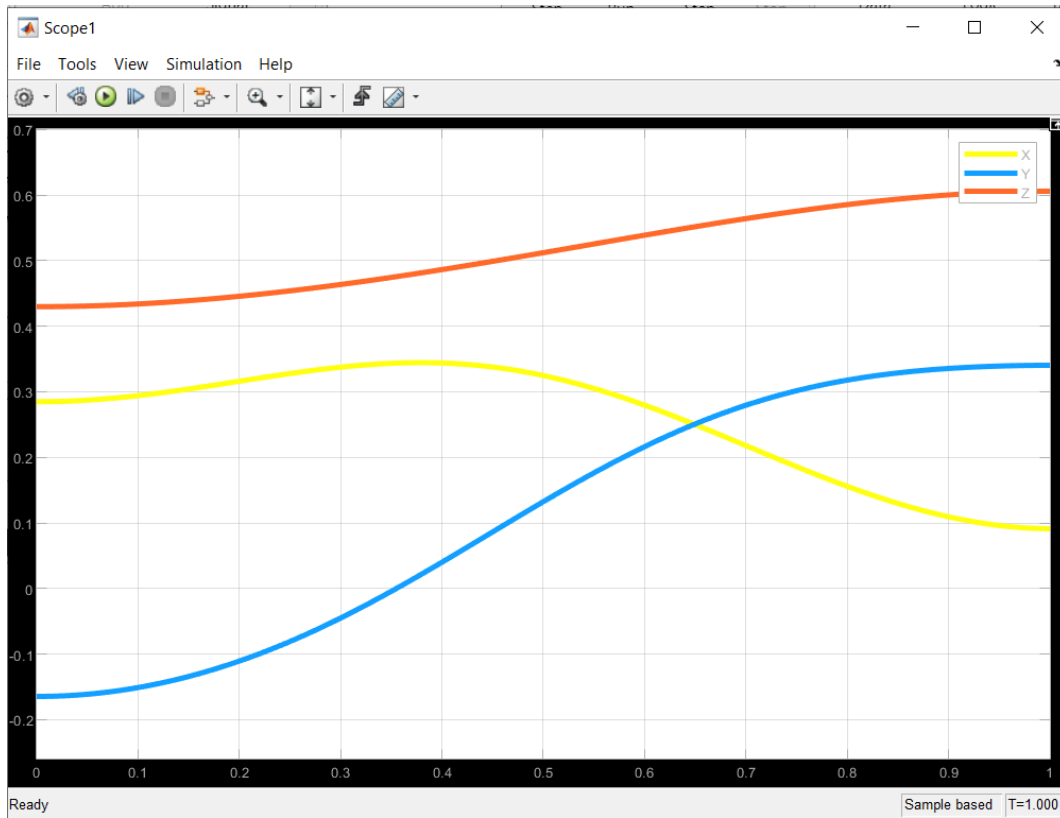


Figure 10: End-effector coordinates X (yellow), Y (blue), Z (orange) versus time (s) for Question 1, showing smooth motion from the start point to the end point.

You can view the smooth motion of the robot in the attached video HW5_Q1.mp4.

2.2 Question 2

In order to answer Question 2, a trapezoidal profile must be designed for the end-effector, and this profile is then used to obtain $s(t)$. Next, using the start and end points, the unit vector of the motion path is constructed, and for every stage of the motion the position traveled along the path is computed, building up a matrix of end-effector coordinates at the different points along the path.

$$V_{\max} = 300 \frac{mm}{s}, \quad a_{\max} = \pm \frac{g}{4}$$

$$t_a = v_{\max}/a_{\max}$$

$$t_d = \frac{\Delta s}{v_{\max}}$$

$$t_f = t_d + t_a$$

Motion stages:

$$\begin{cases} 1) 0 < t < t_a \\ 2) t_a < t < t_d \\ 3) t_d < t < t_f \end{cases}$$

For each of these stages, following the course notes, $s(t)$ is obtained by integrating the trapezoidal velocity profile for that stage. Then, using the relation

$$X_s = X_i + s * unit_x$$

where

$$unit_x = \frac{\vec{X_f} - \vec{X_i}}{|\vec{X_f} - \vec{X_i}|}$$

1

we sweep through the entire motion path and obtain the end-effector coordinates at each specified instant of time.

Then, using inverse kinematics, we obtain the values of the joint variables at each coordinate.

The process implementing all of the operations above is shown in the code below (the trapezoidal-profile design relations are taken from the course notes):

```
v_max = 300;
a_max = (9.81*1000)/4;
delta_s = sqrt(((Xf(1)-Xi(1)))^2+((Xf(2)-Xi(2))^2)+((Xf(3)-Xi(3))^2));

ta = v_max/a_max;
td = delta_s / v_max;
tf = td + ta;

t1 = 0:0.01:ta;
t2 = ta:0.01:td;
t3 = td:0.01:tf;
s1 = 0.5*a_max*t1.^2;
```

¹The original report writes the subscript on the second term as X_a rather than X_i ; this appears to be a typo, since the accompanying MATLAB code computes $unit_x = (Xf - Xi)/delta_s$, i.e. using the same start point X_i as in the X_s formula above. The translation here follows the code.

```

s2 = v_max*(t2-0.5*ta);
s3 = v_max*(t3-td) + a_max*td*(t3 - td) - .5*a_max*(t3.^2-td^2) + v_max*(
    td - .5*ta);

unit_x = (Xf - Xi)/delta_s;
s_len = length(s1)+length(s2)+length(s3)-2;
Xs = ones(3, s_len);

for i = 1:length(s1)
Xs(:,i) = Xi + s1(i).*unit_x;
end

for i = 1:length(s2)
    Xs(:,i+length(s1)) = Xi + s2(i).*unit_x;
end

for i = 1:length(s3)
    Xs(:,i+length(s1)+length(s2)) = Xi + s3(i).*unit_x;
end

Q = ones(3, s_len); %joint variables
for i = 1:s_len
    x = Xs(1, i); y = Xs(2, i); z = Xs(3, i);
    [a,b,c,d,e,f] = InvKin(x,y,z);
    Q(1, i) = a;
    Q(2, i) = c;
    Q(3, i) = e;
end

tt = 0:0.01:tf;
theta1_2 = Q(1, :)*(pi/180);
theta2_2 = Q(2, :)*(pi/180);
dis = Q(3, :)/1000;

```

Then, we turn each of the joint-variable signals into a new scenario (x2) in the Signal Editor block. (The x1 scenario in each Signal Editor block corresponds to Part 1 of the exercise.)

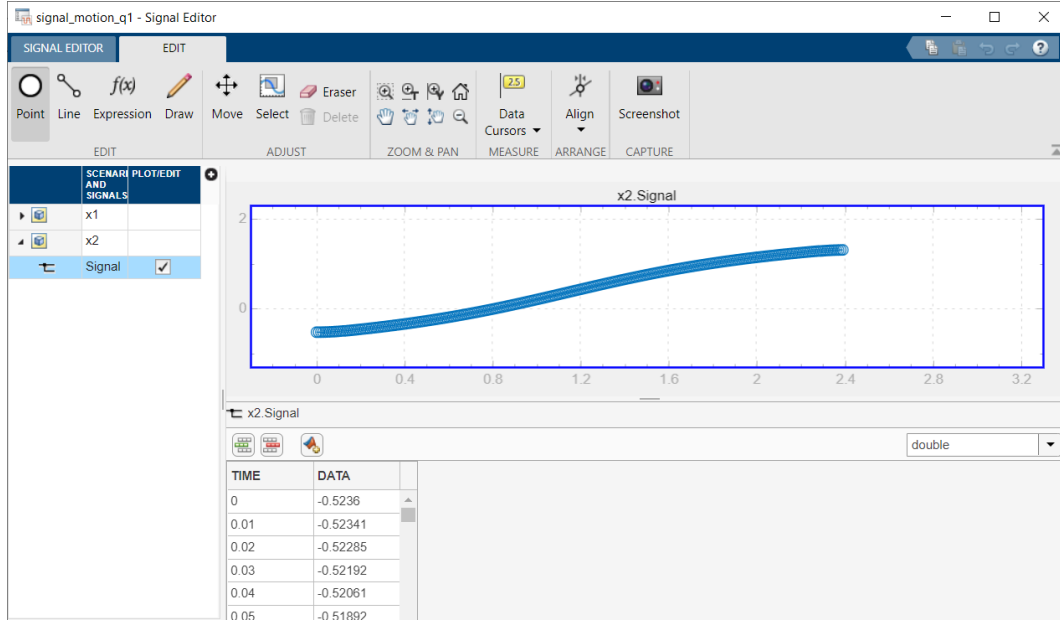


Figure 11: Signal Editor – scenario “x2” for joint 1, showing the new trapezoidal-profile-based trajectory for Question 2 over its (longer) motion duration $t_f \approx 2.4$ s.

Then, we feed the signals to the joint inputs and deliver the output in the second Simulink model file (HW5_Model1.2.slx).

(The only difference between Model 1 and Model 2 is the scenarios selected in the Signal Editor blocks; the block diagram itself is otherwise identical to the one shown in Figures 6 and 8.)

The joint motion curves and the end-effector coordinates are likewise as follows:

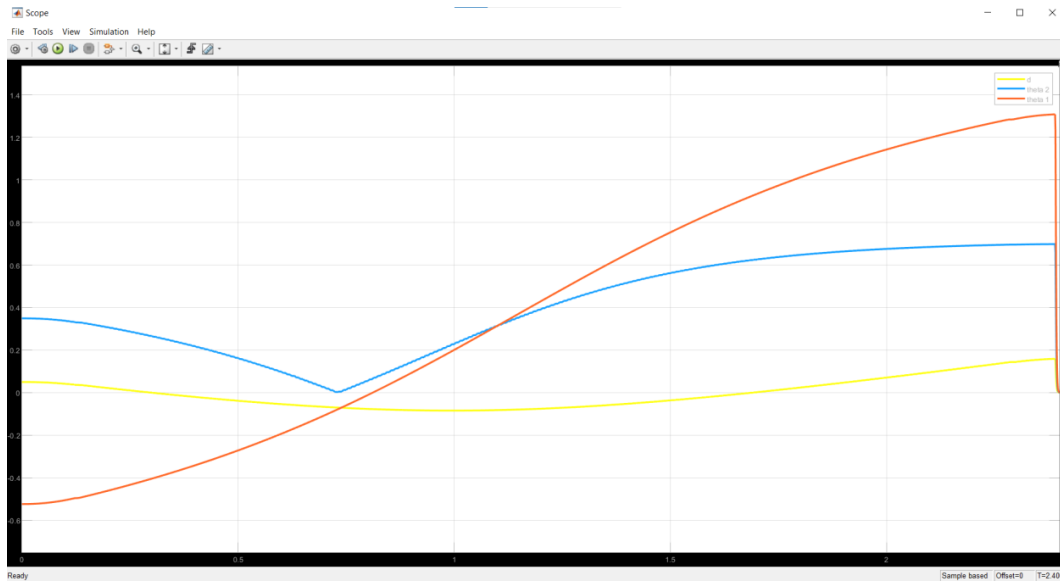


Figure 12: Scope output for Question 2 – joint trajectories θ_1 (orange), θ_2 (blue) and d (yellow) versus time (s), generated from the trapezoidal end-effector speed profile.

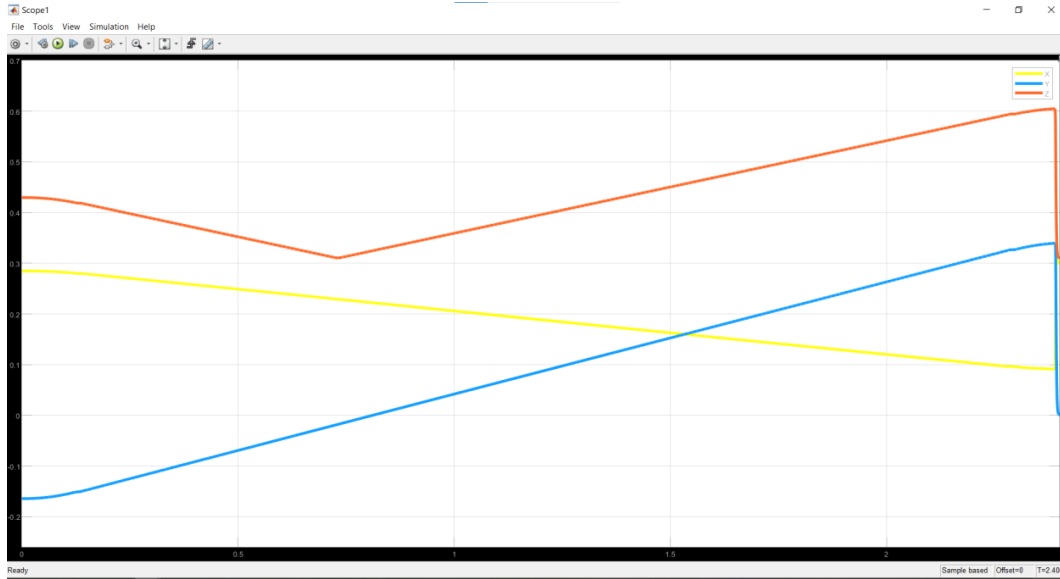


Figure 13: End-effector coordinates X (yellow), Y (blue), Z (orange) versus time (s) for Question 2, showing straight-line motion of the end-effector under the trapezoidal velocity profile.

You can view the motion of the robot for this part in the attached video HW5_Q2.mp4.

The code and models are included as attachments.